

Maîtriser l'Observabilité et la Performance

De FastAPI à Grafana : un guide de projet complet pour construire, tester et optimiser.



Basé sur le brief "Monitoring & Performance - Observabilité et Stress Testing".

Notre parcours d'apprentissage, du concept à l'expertise.

Nous suivrons une approche progressive, construisant notre expertise étape par étape, de la théorie fondamentale à l'optimisation avancée.



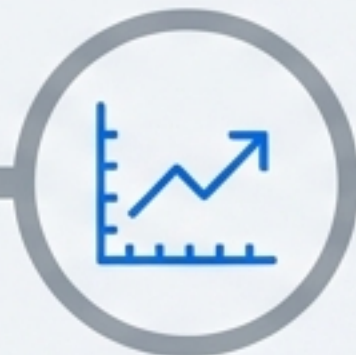
Phase 0 : La
Fondation



Phase 1 : Le
Déploiement



Phase 2 :
L'Instrumentation



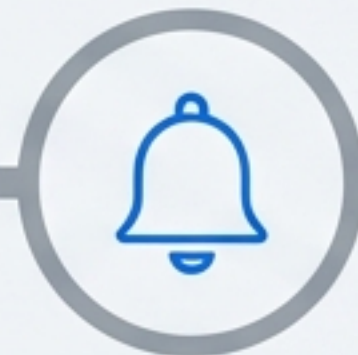
Phase 3 : La
Visualisation



Phase 4 :
L'Épreuve du Feu



Phase 5 :
L'Analyse



Phase 6 :
L'Alerte

Phase 0 : Qu'est-ce que l'observabilité ?

Au-delà du monitoring.

L'observabilité n'est pas seulement la surveillance de métriques connues. C'est la capacité de poser de nouvelles questions sur votre système sans avoir à déployer de nouveau code.

C'est comprendre l'état interne d'un système en examinant ses sorties externes.

Les 3 Piliers



Métriques (Metrics)

Une mesure numérique agrégée dans le temps (ex: latence, taux d'erreur, utilisation CPU). Le 'quoi'.



Logs (Journaux)

Un enregistrement immuable d'un événement discret horodaté. Le 'pourquoi'.

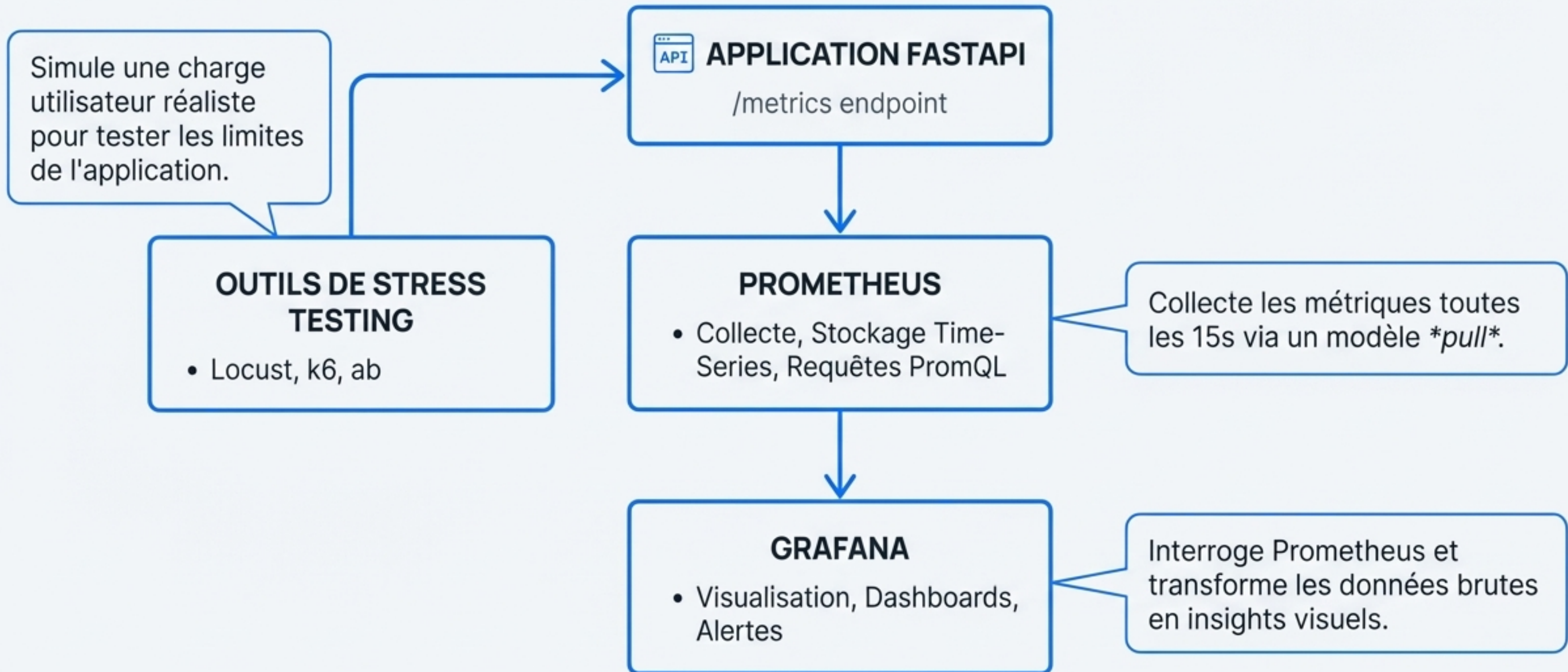


Traces (Traces)

Représente le parcours d'une requête à travers plusieurs services. Le 'où'.

"Your first, last, or everything pane of glass." - Grafana Labs

L'architecture cible : notre stack de monitoring complète.



Phase 1 : Déployer la stack avec Docker Compose.

L'infrastructure en tant que code pour un déploiement reproductible.

Monte le fichier de configuration et un volume pour la persistance des métriques.

Monte les configurations de data sources et de dashboards pour un provisionnement automatique.

```
docker-compose.monitoring.yml

services:
  prometheus:
    image: prom/prometheus:latest
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.retention.time=30d'

  grafana:
    image: grafana/grafana:latest
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin123

networks:
  monitoring:
    driver: bridge
```

Définit le fichier de configuration et la durée de rétention des données (ex: `--storage.tsdb.retention.time=30d`).

Configure les identifiants administrateur et les paramètres de sécurité.

Crée un réseau isolé pour que les services de monitoring communiquent entre eux.

Phase 2 : Au-delà de l'auto-instrumentation : les métriques métier.

Mesurer ce qui compte vraiment pour votre application.

Définition (`app/monitoring/metrics.py`)

```
from prometheus_client import Counter

# Compteur pour les opérations CRUD sur les items
items_created_total = Counter(
    'items_created_total',
    'Total des items créés avec succès.'
)
```

Un `Counter` est une métrique cumulative qui ne fait qu'augmenter, idéale pour compter les événements comme la création d'objets.

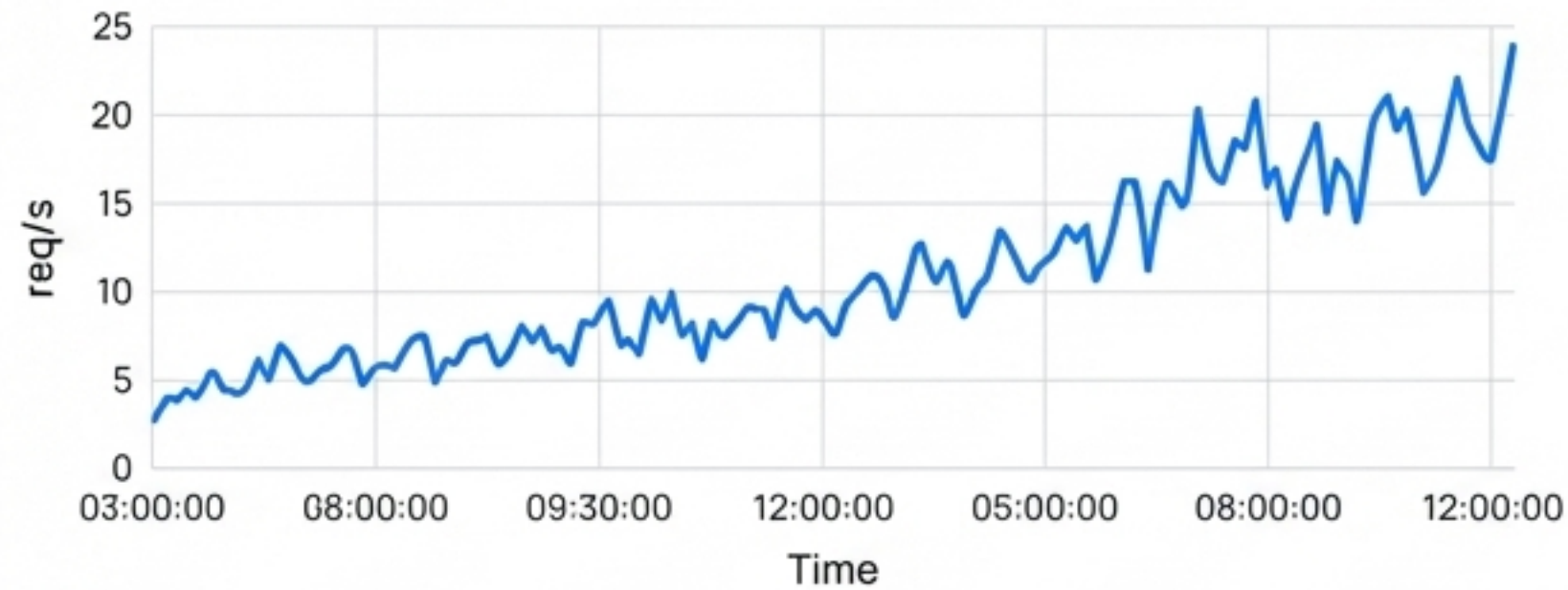
Utilisation (`app/routes/items.py`)

```
@router.post("/")
def create_item(item: ItemCreate, ...):
    new_item = ItemService.create(db, item)
    items_created_total.inc() # Incrémenter après succès
    return new_item
```

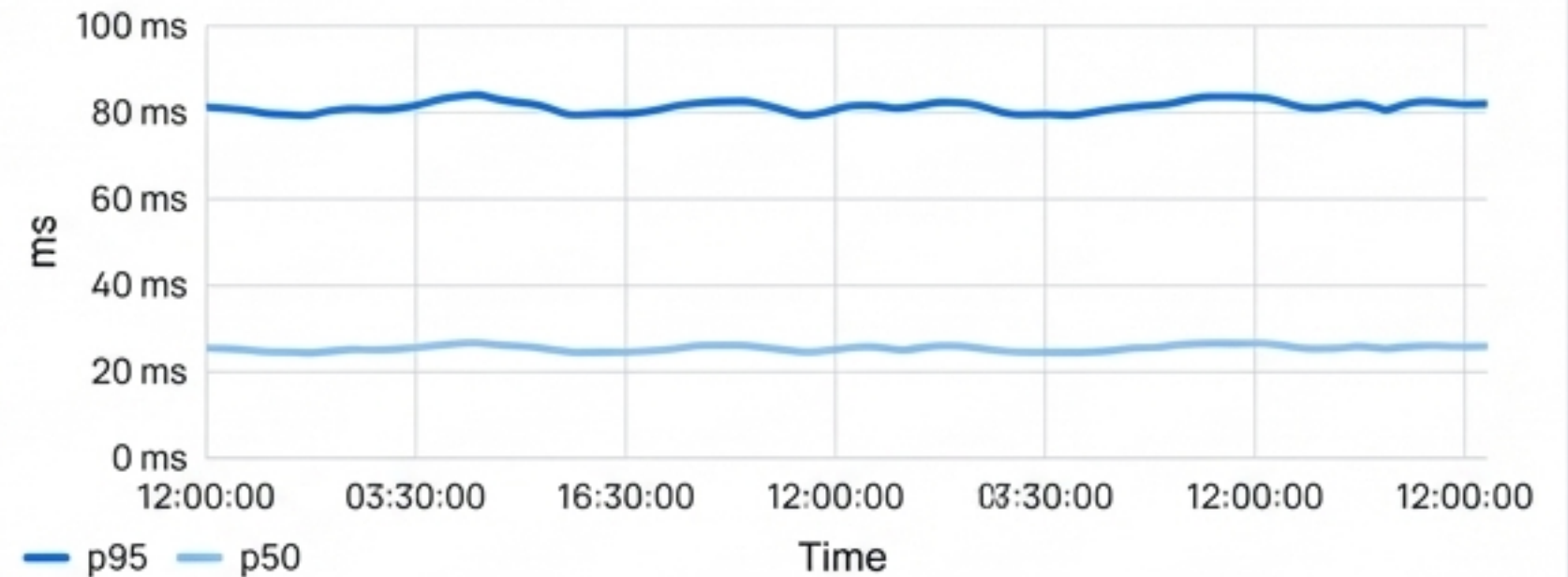
Incrémentez le compteur **après** la réussite exx compteiteur la réussite de l'opération pour garantir que seules les transactions valides sont comptées.

Phase 3 : Donner vie aux données avec Grafana.

Request Rate (req/s)



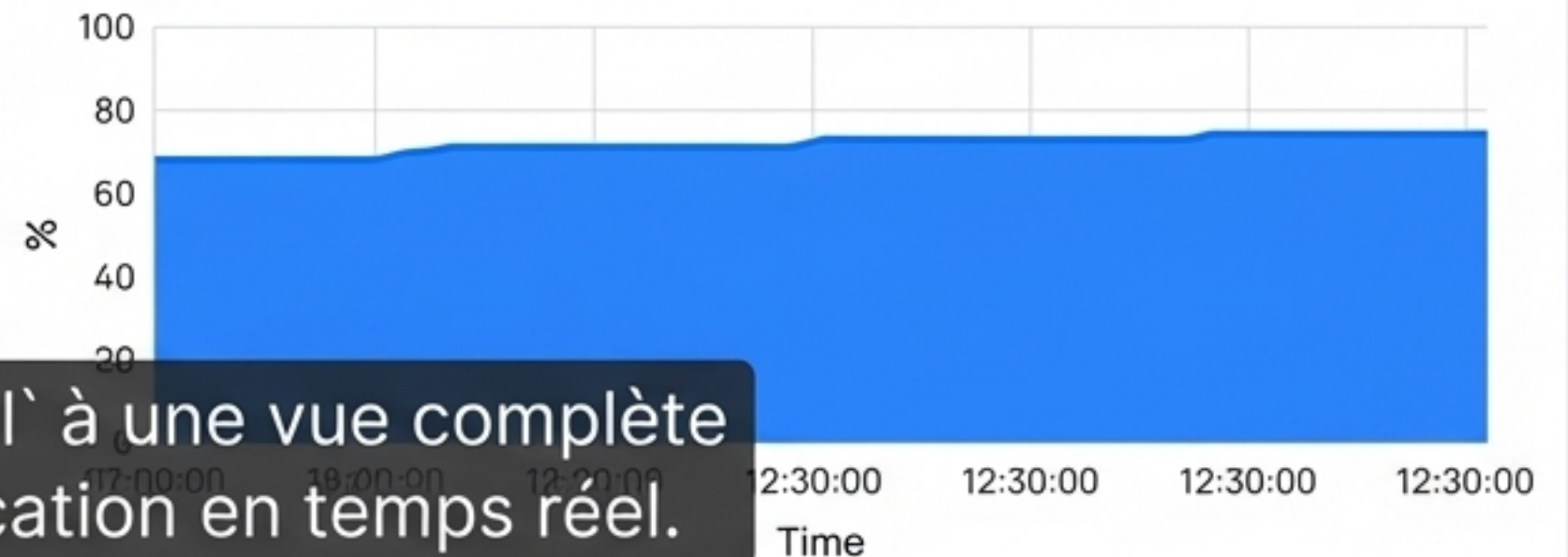
Response Time (Latency)



CPU Usage (%)



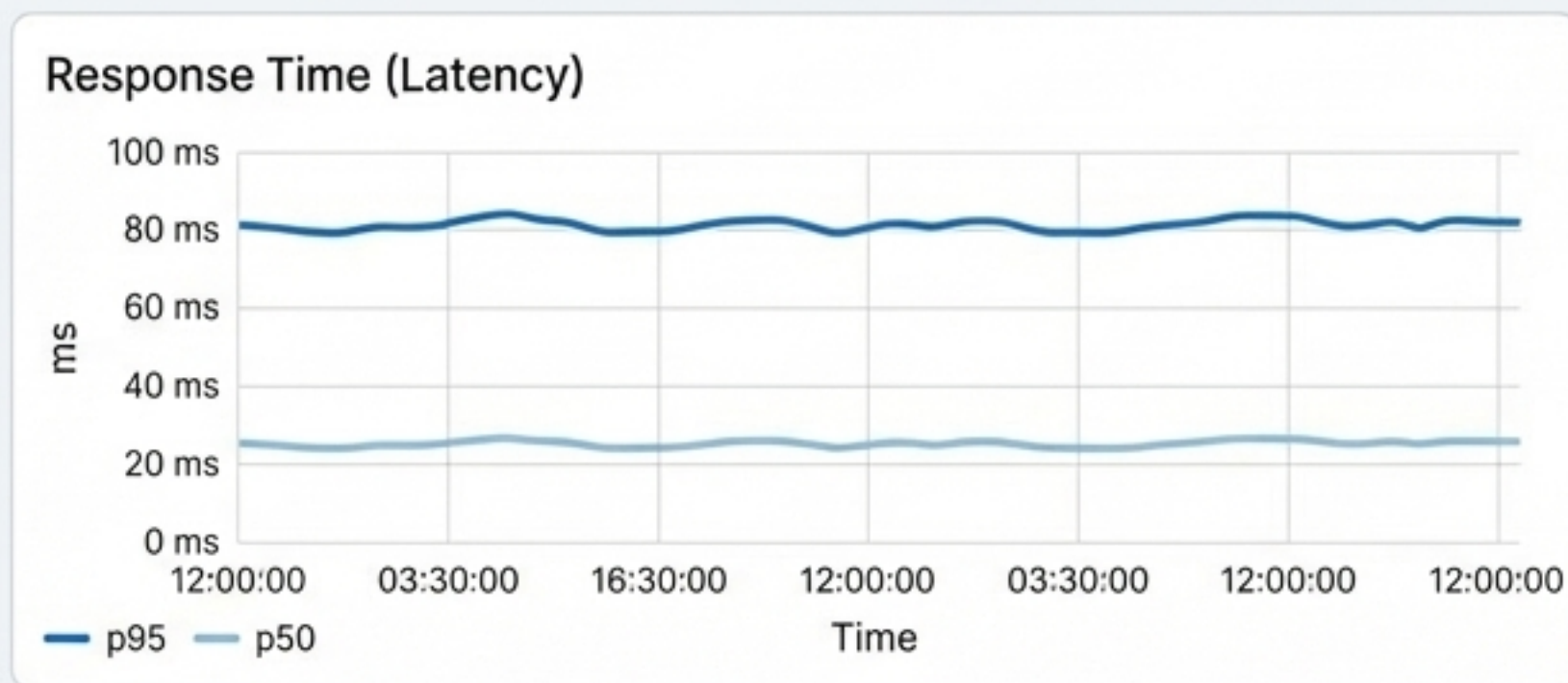
Memory Usage (%)



De `http\_requests\_total` à une vue complète de la santé de l'application en temps réel.

L'art de la requête : introduction à PromQL.

Comment Grafana interroge Prometheus pour créer des visualisations puissantes.



Calcul du 95ème percentile de la latence

```
histogram_quantile(0.95, rate(request_duration_seconds_bucket[5m]))
```

Calcule le 95ème percentile à partir de la distribution fournie par les buckets. En clair : "95% des requêtes ont été plus rapides que cette valeur."

Calcule le taux de croissance par seconde des requêtes sur les 5 dernières minutes pour chaque bucket d'histogramme.

La métrique brute de type `Histogram` collectée depuis l'application.

Phase 4 : L'épreuve du feu, le Stress Testing.

Pousser l'application dans ses retranchements pour découvrir ses véritables limites.

Pourquoi tester ?



Identifier les goulots d'étranglement (CPU, RAM, base de données).



Valider la capacité de charge (combien d'utilisateurs simultanés ?).



Prévenir les pannes en production en comprenant les points de rupture.

Nos Outils de Prédilection

Outil	Langage	Avantages	Cas d'usage
Locust	Python	Interface web, scénarios complexes scriptables, distribué	Simuler des parcours utilisateurs réalistes et complexes.
k6	Go/JS	Très performant, moderne, intégration CI/CD facile	Tests de charge haute performance et scripting moderne.

La Méthodologie

Pendant chaque test, nous observerons nos dashboards Grafana en temps réel pour corrélérer la charge appliquée avec la performance du système.

Simuler un comportement utilisateur avec Locust.

Un exemple de scénario de test pour notre API.

Simule un temps de réflexion réaliste pour un utilisateur entre chaque action.

```
from locust import HttpUser, task, between

class ItemsAPIUser(HttpUser):
    • wait_time = between(1, 3) # Temps d'attente entre les tâches

    @task(5) # Poids 5: tâche la plus fréquente •
    def read_items(self):
        """GET /items - Simule la lecture de la liste d'items."""
        self.client.get("/items")

    @task(2) # Poids 2: moins fréquent
    def create_item(self):
        """POST /items - Simule la création d'un item."""
        item_data = {"nom": "Test Item", "prix": 99.99}
        self.client.post("/items", json=item_data)

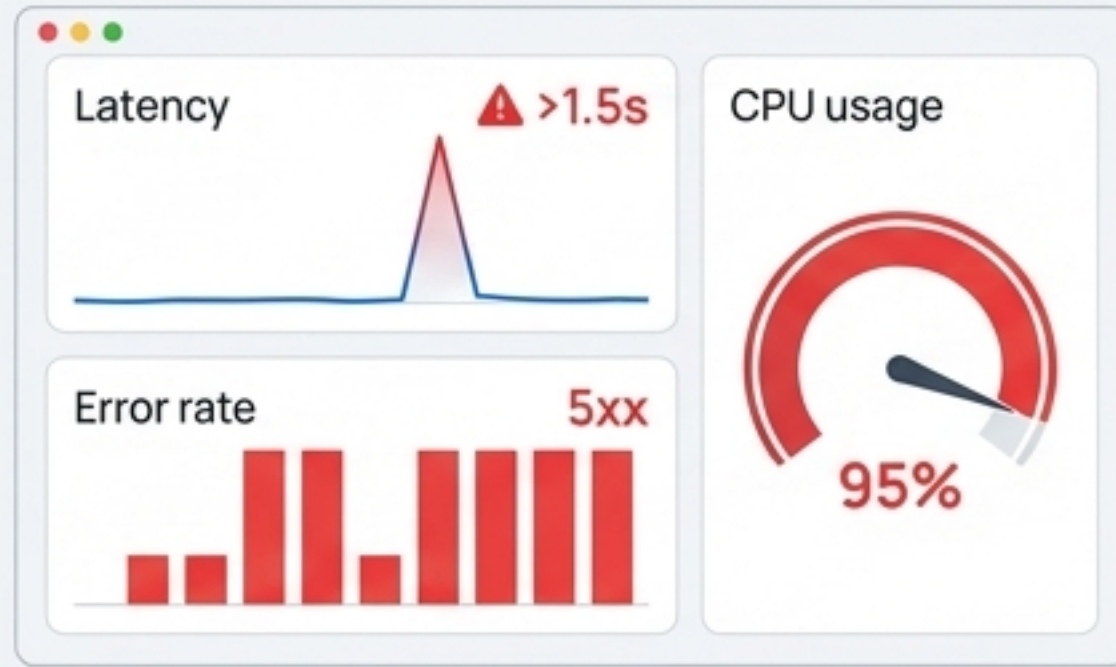
    @task(1) # Poids 1: tâche la plus rare
    def delete_item(self):
        """DELETE /items/{id} - Simule la suppression d'un item."""
        if self.item_ids:
            item_id = self.item_ids.pop()
            self.client.delete(f"/items/{item_id}")
```

Le décorateur `@task` définit une action utilisateur. Le poids (ex: 5) détermine sa fréquence relative. Ici, la lecture est 5 fois plus probable que la suppression.

Phase 5 : Analyser, Optimiser, et Mesurer à Nouveau.

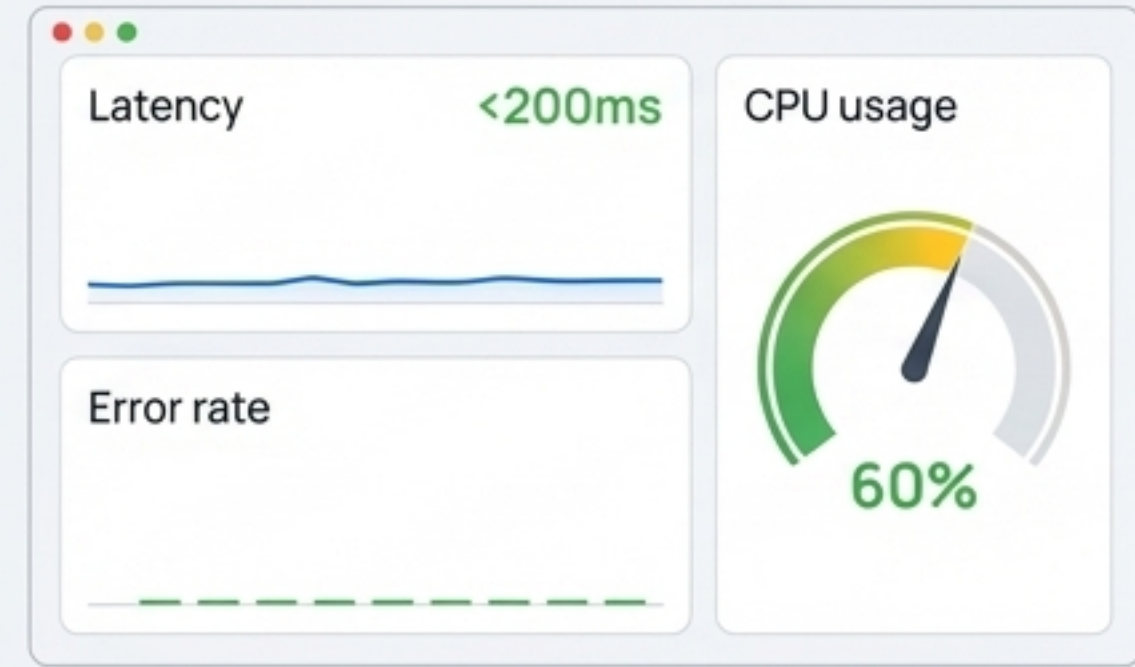
Transformer les données en actions concrètes.

Pendant le Stress Test - Avant Optimisation



****Goulot d'étranglement identifié****: Le pool de connexions à la base de données est saturé, créant une file d'attente et des timeouts.

Après Optimisation



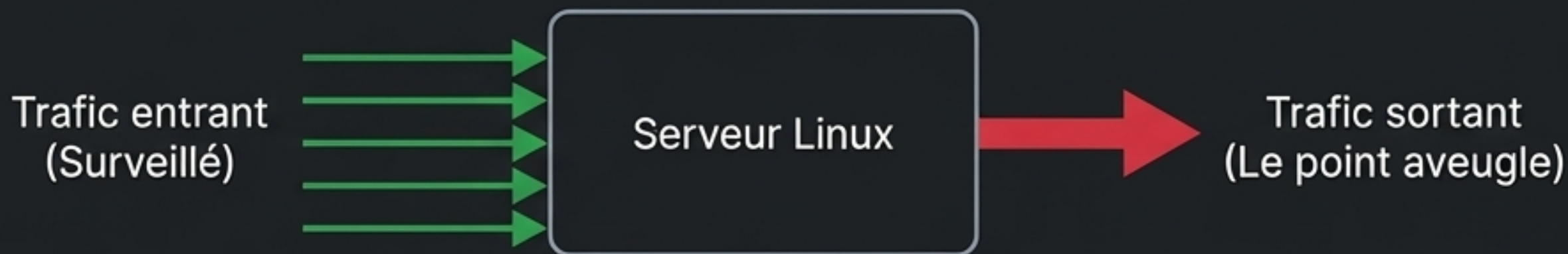
****Optimisation appliquée****: Augmentation de ``pool_size`` et ``max_overflow`` dans la configuration SQLAlchemy de la base de données.

RÉSULTAT

+150% de requêtes par seconde gérées
avec une latence P95 divisée par 8.

Angle Mort : L'Observabilité au service de la Sécurité.

Pourquoi la surveillance des connexions sortantes est votre meilleur signal de compromission.

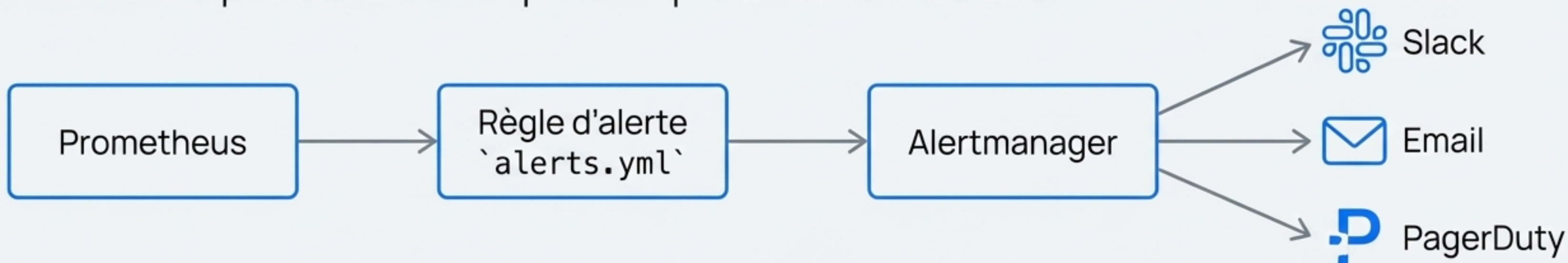


> *"Dans presque tous les incidents sur lesquels j'ai travaillé, l'indicateur le plus clair n'était pas la façon dont l'attaquant *est entré*. C'était la façon dont le serveur a commencé à *répondre*."*

- Un serveur compromis est rarement silencieux. Il contacte son serveur de commande et contrôle (C2), télécharge des charges utiles, ou exfiltre des données.
- **Signaux d'alerte*** Des processus comme ``bash``, ``python``, ou ``cron`` qui initient des connexions réseau vers des IP inconnues.
- **La commande qui révèle tout:** ``ss -tunap`` permet une visibilité immédiate sur ce comportement.

Phase 6 : Devenir Proactif avec l'Alerting.

Détecter les problèmes avant qu'ils n'impactent vos utilisateurs.



Exemple Règle : Taux d'erreur élevé

```
- alert: HighErrorRate
  expr: (sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m])) > 0.05
        / sum (http_requests~==n1))
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Taux d'erreur élevé sur le service FastAPI"
    description: "Plus de 5% des requêtes ont échoué sur les 5 dernières minutes. Valeur actuelle : {{
      $value | humanizePercentage }}."
```

La clause `for` est cruciale pour éviter la 'fatigue d'alerte'. L'alerte ne se déclenche que si la condition est vraie pendant 5 minutes consécutives.

Principes clés pour une observabilité réussie.

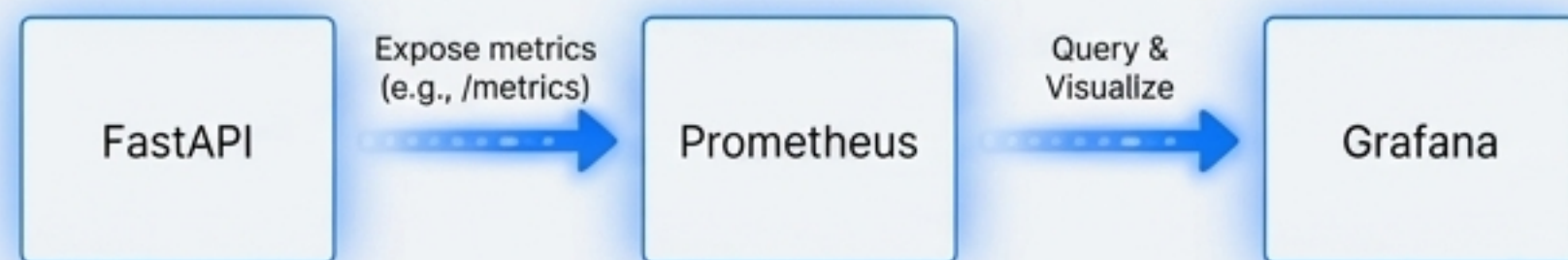
À Faire (Do's)

- ✓ **Mesurer avant d'optimiser:** Ne pas optimiser prématurément. Utilisez les données pour guider vos efforts.
- ✓ **Corréler les métriques:** Observez vos dashboards *pendant* les tests de charge pour comprendre les relations de cause à effet.
- ✓ **Instrumenter le métier:** Les métriques techniques sont utiles, mais les métriques métier (items créés, commandes passées) apportent la vraie valeur.
- ✓ **Versionner vos dashboards:** Exportez vos configurations Grafana en JSON et commitez-les avec votre code.

À Éviter (Don'ts)

- ✗ **La haute cardinalité:** Évitez d'utiliser des labels avec des valeurs uniques (ex: user_id, request_id) dans les métriques Prometheus. Cela peut surcharger la base de données.
- ✗ **La fatigue d'alerte:** Ne pas alerter sur tout. Définissez des seuils pertinents et utilisez la clause `for` pour ne notifier que des problèmes persistants.
- ✗ **Tester en production:** Créez un environnement de staging dédié et aux ressources similaires pour des tests fiables.

La boucle est bouclée : de l'application au système observable.



Nous avons déployé une stack complète, instrumenté notre application, visualisé sa performance, identifié et corrigé ses goulots d'étranglement, et mis en place des alertes proactives.

Prochaines Étapes ('Frontières à explorer')

- **Tracing distribué** avec OpenTelemetry pour suivre une requête de bout en bout.
- **Logging structuré** avec Loki pour agréger et interroger les logs de manière centralisée.
- **Définition de SLOs** (Service Level Objectives) pour formaliser vos objectifs de performance et de disponibilité.

"You can't improve what you don't measure."

— Peter Drucker